CSIS – 4440 – 003 Mobile CyberSecurity Project Proposal



Mobile Cyber

FORENSICS

Presented by GROUP4

Team Members:

(Team Lead) Daniel Anaya Murcia – 300403740

Steven Anderson – 300414292

Tools & Apps Overview – Daniel

For my part of the project I will use one forensic tool that we have not covered in class, **ALEAPP**, to analyze two Android apps that are not on the course list, **Firefox for Android and Joplin**. I chose these apps because they do not need a phone number, they keep their data in local files that a forensic tool can read, and ALEAPP already has parsers for both of them. I will use the same test device and the same scenario as Steven, so my results can be connected to his work (see Integrated Component).

ALEAPP – Primary Tool



ALEAPP (Android Logs, Events And Protobuf Parser) is an open-source Python tool created by Alexis Brignoni and the digital forensics community (Brignoni, n.d.). It reads an extraction of an Android device, for example a folder, a zip or tar file, or a disk image, and looks for the files of many apps and system services. Each app has its own parser that reads those files and writes the results to HTML and TSV reports, so the examiner does not have to open every database by hand. ALEAPP also collects the results of its parsers into a timeline. Recent versions open the databases in read-only mode, which helps to avoid changing the evidence. I will use the current release and write its exact version in the report.

Firefox for Android – Web Activity Forensics



Firefox for Android is Mozilla's mobile browser (Mozilla, n.d.). It saves browsing history, page visits, bookmarks, search terms, downloads, cookies, form entries, site permissions and recently closed tabs in files inside the app folder, mostly in a SQLite database called places.sqlite. ALEAPP has a parser for each of these.

Forensic Relevance: A browser shows which links a user opened and at what time, which is useful when a link was received in a message. Firefox is built on Mozilla's Gecko engine, so its tables and timestamps are different from the Chromium layout used by Chrome, which is on the course list.

Proposed Forensic Activities: I will open test links that Steven sends through Discord, run a few searches, download a harmless test file, and then clear the browsing data. I will acquire the device before and after clearing the data, and compare what ALEAPP reports with what is still inside the database files.

Joplin – Notes and Deletion Forensics



Joplin is an open-source note-taking app (Cozic, n.d.). It stores notes, to-do lists, notebooks and attachments in a local SQLite database called joplin.sqlite. For each note, the ALEAPP parser reports the title, body, notebook, created, updated and deleted times, and the location when the note has one.

Forensic Relevance: People often keep passwords, recovery codes and reminders in a notes app. A public CyberDefenders lab about an Android data breach is built around an employee who stored credentials in a notes app (CyberDefenders, n.d.), so this is a realistic place for an examiner to look.

Proposed Forensic Activities: I will create notes with made-up credentials and recovery codes, edit some of them and delete others. Then I will check what ALEAPP shows for the deleted notes and whether anything remains in the database file after the deletion.

Forensic Scenario

To make the analysis realistic, Steven and I will both work around one scenario. A gamer says that their gaming account was accessed after a conversation on Discord. In our controlled version, a test account receives a suspicious link in Discord, opens it in Firefox, saves a made-up password and recovery code in a Joplin note, and approves or rejects a Steam Guard login request. At the end, the user deletes the Discord messages, clears the Firefox history and deletes the note to hide what happened. The examiner then has to rebuild the order of events and say which evidence is still available. We will only use dummy accounts, made-up credentials and harmless test links.

Novelty and Value Addition

ALEAPP already gives parsed tables for these apps, so the value I want to add is in how the results are checked and used. First, I will keep a written action log with the exact time of every action in the scenario and compare it with what ALEAPP reports. This will show what the parsers find, what they miss, and where timestamps are wrong, especially after the delete and clear steps. Second, I will compare the acquisitions before and after deletion to see what ALEAPP can and cannot report once the user tries to hide the activity.

Third, as far as I can tell from reading the ALEAPP source code, its timeline stores each row under the first column of the artifact only, so the updated and deleted times of a Joplin note do not appear as their own events. My script, described in the Integrated Component section, will list every date as a separate event.

I will test ALEAPP and both apps first on the emulator. If one app does not work with ALEAPP on our Android version, I will replace it with another app that has an ALEAPP parser, such as OpenTracks or Aegis Authenticator.

Tools & Apps Overview – Steven

This project aims to explore and apply mobile forensic tools to investigate digital artifacts generated by two Android applications: **Steam Mobile and Discord**. The investigation will focus on identifying locally stored evidence, analyzing user activities, and examining how application data changes or persists after selected actions.

The project will cover two different forensic scenarios: authentication-related activities in Steam Mobile and communication-related activities in Discord. Controlled experiments will be conducted to determine what information can be recovered and how the collected evidence can support forensic analysis.

Additionally, the project will explore differences in forensic analysis between a rooted physical Android device and a rooted Android emulator, subject to device compatibility and feasibility.

Steam Mobile – Authentication Forensics



Steam Mobile is an Android application developed by Valve Corporation that allows users to access and manage their Steam accounts. It provides features such as browsing the Steam Store, managing account information, viewing community activities, and accessing Steam Guard for two-factor authentication (2FA). Steam Guard also supports QR-code login, sign-in confirmations, and authentication codes to help protect user accounts.

Forensic Relevance: The application is particularly interesting for investigating authentication-related activities, account access, and locally stored security-related artifacts.

Primary Tool: Objection

Objection is a mobile application exploration toolkit built on Frida that supports dynamic runtime analysis. It will be used to investigate Steam Mobile's application behavior and accessible local storage during authentication-related activities.

The investigation will focus on Steam Guard, including opening the authenticator, approving or rejecting login requests, and logging in or out of the application. The objective is to determine whether these actions produce identifiable local artifacts and whether the artifacts can provide useful forensic evidence of account activity.

Supporting Tool: JADX

JADX is an Android application decompiler that allows examination of application code and structure. It will be used to investigate Steam Mobile's implementation of local data storage and help identify potentially relevant files, databases, and application components.

Proposed Forensic Activities:

- Examine locally accessible application data before and after authentication activities.
- Investigate changes associated with Steam Guard login approvals and rejections.
- Identify potential timestamps, metadata, and application storage artifacts.
- Examine whether relevant artifacts remain after logout or other application actions.

Discord – Communication and Data Persistence Forensics



Discord is a communication platform that allows users to interact through text messaging, voice calls, video calls, and media sharing. Originally popular among gaming communities, Discord supports servers, channels, direct messages, and group communication. Its Android application is built primarily using React Native and synchronizes communication data across devices.

Forensic Relevance: Discord provides opportunities to investigate communication-related artifacts, such as messaging metadata, cached media, application activity, and the persistence of locally stored data following user actions such as message deletion.

Primary Tool: SQLite4N6

SQLite4N6 is a forensic analysis toolkit for examining SQLite database structures and potentially recoverable records. It will be used to investigate Discord's locally stored SQLite databases, if available, including database records, metadata, write-ahead logs, and potentially deleted or historical data.

The investigation will focus on identifying communication-related evidence and examining how locally stored data changes following user activities.

Supporting Tool: Diffoscope

Diffoscope is a file comparison tool capable of identifying differences between files and directories. It will be used to compare acquired Discord application artifacts before and after controlled activities to identify changes in stored data.

Proposed Forensic Activities:

- Send and receive messages using controlled test accounts.
- Edit and delete selected messages.
- Upload and download media attachments.
- Examine accessible databases, cached files, and relevant metadata.
- Investigate which artifacts remain after message deletion, cache clearing, or logout.

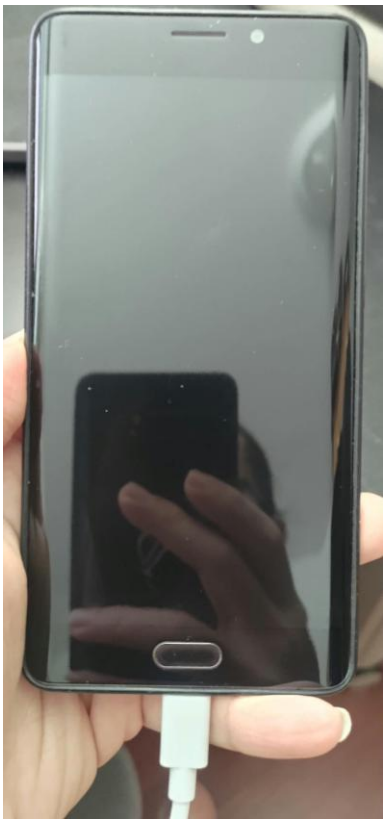
Supporting Tools

These supporting tools will complement the primary forensic analysis tools.

- **Android Debug Bridge (ADB)** will be used to communicate with the Android test environments; access permitted device information and acquire available application files for forensic examination.
- **AndroidQF (Android Quick Forensics)** may also be explored as an additional acquisition tool to collect system-level forensic artifacts and support evidence collection.

Proposed Forensic Environment

The project will explore two Android testing environments:



1. Rooted Physical Android Device

A Xiaomi Mi Note 2, previously configured with a custom Android ROM believed to be LineageOS and rooted using Magisk, is proposed as the physical testing device.

The device will first be evaluated to confirm its installed Android version, root accessibility, application compatibility, and operational stability. Its battery condition will also be checked because the device has experienced battery drain.

2. Rooted Android Emulator

An Android Studio emulator configured with an Android system image supporting root access will be used as an alternative and controlled testing environment.

The emulator will allow repeatable experiments and provide a fallback if the physical device cannot support the required applications or forensic tools.

If both environments are suitable, equivalent experiments will be performed to compare forensic artifact availability and acquisition limitations.

Novelty and Value Addition

The proposed project aims to extend beyond basic Android application data extraction by exploring multiple forensic tools and applying different investigative techniques to two applications with distinct purposes.

The project's proposed novel aspects include:

- Investigation of different forensic scenarios: Steam Mobile will be examined for authentication-related artifacts, while Discord will be investigated for messaging, media, and data persistence.
- Dynamic application analysis: Objection will be explored to examine runtime behavior and accessible application storage during controlled Steam Guard authentication activities.
- Advanced SQLite forensic analysis: SQLite4N6 will be used to examine database structures, write-ahead logs, and potentially recoverable historical or deleted records beyond conventional database viewing.
- Before-and-after artifact comparison: Application evidence will be collected at different stages of controlled activities to identify changes in locally stored information.
- Physical device versus emulator comparison: If compatible, a rooted Xiaomi Mi Note 2 and rooted Android emulator will be used to investigate differences in forensic artifact accessibility,

storage behavior, and evidence collection. Differences in Android versions, device configurations, and application versions will be documented when interpreting the results.

- Forensic evidence validation: Recovered artifacts will be compared against documented test activities to evaluate the accuracy and reliability of forensic findings.
- Understanding forensic limitations: The project will examine how Android security restrictions, application architecture, authentication protections, and storage mechanisms affect evidence accessibility and interpretation.

Integrated Component

Steven and I plan to run the scenario above on the same test device. We will start with the rooted emulator and also use the Xiaomi Mi Note 2 if all four apps work there. Every action will be written in a shared action log with its time, so we have a known answer to compare with. After each stage we will acquire the device with ADB, and with AndroidQF if Steven can use it, and use the same extraction for both parts of the project. I will run ALEAPP on it, and Steven will analyze the Steam Mobile and Discord data with his tools.

The two parts fit together because ALEAPP and Steven's tools cover different things. As far as I could see in its source code, ALEAPP has no Steam parser, so Steven's Steam Guard findings will fill that gap. On the other hand, ALEAPP has parsers for Discord, so I can compare its results with what Steven finds using SQLite4N6. If time allows, we will also run SQLite4N6 on my Firefox and Joplin databases after the deletion step, to see if it recovers records that ALEAPP does not show.

To combine everything, I will write a small Python script that reads the ALEAPP output, a simple CSV file with Steven's findings (time, app, event and source) and the action log. The script converts all times to UTC, puts every event in one timeline sorted by time, and marks events from different apps that happen within a few minutes of each other, for example a Discord message followed by a Firefox visit and then a Steam Guard approval. It will also list the actions from the action log that no tool found, such as deleted items. The result will be one integrated HTML and CSV report for the whole scenario, which we will use in the demo.

AI Use

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used
OpenAI Codex Used by: Steven	Version: 5.6, Plus	1. Searching free forensic tools appropriate for proposed apps. 2. Brainstorming which tools to use, project idea and possibility of using physical android device. 3. Finding out if both apps can work in rooted devices or not. 4. Help creating project proposal documentation. 5. Create logo for project proposal.
Claude Used by: Daniel	Version: Sonnet 5.5, Pro	1. Comparing possible forensic tools and apps from the Blackboard guideline documents, and explaining what each tool does and what it needs.

		2. Recommending a list of apps that meet the guidelines, and checking the ALEAPP source code and release notes to see which app parsers exist. 3. Helping to consolidate my idea for integrating Steven's work and mine into one shared scenario (Integrated Component). 4. Wording this part of the proposal, correcting spelling, drafting the email to the professor, building a work schedule, and setting up the GitHub repository from the course guidelines.

Value Additions (Sync with the points stated on the table):

1. Explore and research a little bit regarding the tools that ChatGPT provided to me and picking the one I feel confident and suitable for the project.
2. Asking why ChatGPT recommend those tools. Scoping the project. Adding my rooted phone for testing possibility.
3. Basically, like point explained above, because I believe some apps literally can't be opened on rooted phone (like banking apps). I want to know if both apps can be opened on rooted device or not.
4. It's for saving time especially for doing references. I'm doing the document styling and copy the words after verifying it.
5. I'm not adding anything else to the prompt. Just asking to generate a logo that suits the theme of the project.

Value Additions – Daniel (Sync with the points stated on the table):

1. I researched the different types of forensic tools myself. This gave me the knowledge to understand what I wanted to do and to choose ALEAPP. I organize my ideas, and the AI helps me write and word them better.
2. The AI recommended a strong list of apps, but I chose Firefox for Android and Joplin because they do not have many bugs, which is a common problem with mobile apps.
3. I proposed the original idea for the integration of Steven's tool and mine. The AI only helped me consolidate it into the Integrated Component.
4. I read and edit every text the AI writes before it goes into the document, and I keep the final decisions about the tools, the apps and the plan.

Appendix

Steven's Prompt History

Thu, Oct 15 10:12 PM

Expected Hours
Each student is expected to put in at least 60 hours towards the project through out the term. This time may likely be more depending on your project needs, skill-level and any other issues that might arise in an open-ended project. Your work logs (see deliverables in progress report, midterm report and final report) must reflect this amount of time.
Project Goals
You will learn to use and apply NOVEL FORENSIC TOOLS and PERFORM FORENSIC ANALYSIS on NOVEL APPS. The goal is to do background research, demonstrate self-directed learning and application, and illustrate these learnings and application with HANDS-ON final demonstration.
Choosing projects
For EACH MEMBER in your group:
Option 1: Existing forensic tool not explored in class + forensic analysis on user applications not explored in class.
You will explore, learn and apply at least ONE NOVEL forensic tool/student AND perform forensic analysis on at least TWO novel user applications for forensic analysis. Doing analysis across multiple apps to examine user/events/timeline discovery correlated across user applications will be an added enhancement.
Option 2: New forensic tool creation
Alternatively, you can create a new forensic tool (e.g., python script suite or mobile application) that performs forensic analysis and/or generation of forensic reports on user applications and system information on mobile devices.

Here is a tentative list of tools and apps that will be discussed this term:

adb
adb backup extractor
Magnet Acquire
Autopsy
Carbon
HBE
Helium Android
Nandump
FTK Imager
Android Backup Extractor
DCode
DB Browser Lite
JsonViewer
vtuploader
yara
hasher
nmap - network mapper

Contacts and call analysis
MMS and SMS databases
User Dictionary Analysis
Gmail
Chrome

Decoding webkit time frame
Maps
Keep
Facebook
Facebook Msnger
Skype/Teams
Snapchat
Viber
Tango
Whatsapp
Kik
Wechat
Malware - intro and detection

For the NOVEL aspects of your group project, you may pick up any other tool or apps NOT on this list. If you would like to still use any of these, that is fine as long as there are NOVEL parts to your project.

so since this is regarding mobile cybersecurity. I don't really know any forensic tools, can you give me examples? my team mate using aleapp or mobsf

for the apps I was thinking of testing steam and discord. what are a good project idea for both application?

can you give me more details regarding first project you suggest

I was thinking of identifying them separately instead, is that better?

I don't really know if I'm gonna test it in simulated rooted device or not, I will ask the prof later. but will there be any difference whether I do the forensic on rooted/ non rooted device?

but can discord and steam be installed on rooted device?

but isn't doing forensic on rooted device defeat the purpose of doing forensic itself?

are you able to access video? I have a recording regarding the project.

the video is 2+ hours long since it recording the whole lecture session, I dont know if I can upload it here

this is the transcript. the transcription is not 100% able to identify her voice since she use laptop mic as the voice input.
[Attachment: en-4440_Lec4Recording.txt]

this is the project guideline. what do you think compared to what was in the transcript?
[Attachment: 4440F26ProjectGuidelines_Slides.pdf]

but I think steam mobile app is mostly used for the steam guard access rather than browsing game and other stuffs. is it still ok to use steam as apps candidates?

but if I'm testing steam guard using dummy account there's some experiment that's limited such as trading in steam market.

there is another limitation, I don't know if emulated android can do QR scan on steam mobile

can you suggest me more forensic tools? maybe we can use different forensic tools for the apps since I'm not limited to one forensic tool right

Friday, Oct 16 10:01 AM

> I'm going to give the proposal to prov, so what's your proposal for the tools for each application and what we gonna use the tools for.

Friday, Oct 23 1:46 PM

> can you generate a logo for the project proposal. make the one suit the theme of mobile cybersecurity
>
> I have verified with the professor that we gonna use rooted device for the project. I have an old xiaomi note 2 rooted, I was thinking to use that instead of an emulator. Do you think it will add novelty?
>
> I have xiaomi mi note 2, I rooted it to change the chinese rom into lineage os iirc. It's been awhile already since I checked it. Do you have any other question?
>
> Here are my Xiaomi Mi Note 2 details for the Steam Mobile and Discord forensic project:
> Android version: unknown
> Root status: Magisk
> Device usage: dedicated
> Testing environment preference: comparison
> Additional information: Like I said, I haven't boot it for awhile since the battery is leaking. Right now I'm doing the project proposal, can you help me with the documentation? I'm doing the tools and apps overview and have to list :
> - Tools and Apps: Provide a list of tools and apps that student is proposing to explore, understand and apply
> - Novelty: Highlight as bullet points or as a paragraph on what is the novelty of the proposed project. How will you add value over and above what the tool already provides. This is especially encouraged as an option to have in cases where the tool and application analysis is fairly straightforward
>
> Please assess compatibility, recommend a forensic setup and next steps.
>
> by leaking I mean it drained the battery faster than usual. there is no sign of visual bump from the phone.
>
> can you rewrite the whole proposal for me to copy, I can verify it easier that way.
>
> can you give me quick overview of what steam and discord is about. I want to add apps overview.
>
> can you list all my prompt for this chat and their time stamp for me to put in document appendix

Sat, Oct 24 1:02 AM

> can you give me references for all the tools and apps for project? objection, jadx, sqlite4n6, diffoscope, adb, androidqf, steam guard, discord.

Daniel's Prompt History

Many of my queries were written in Spanish, which is my first language. Because the instructor reads English, the entries below are an English summary of everything I asked the AI during the project. They are summaries, not word-for-word copies, and each entry is in date order.

Fri, Oct 9

> My first query. I asked the AI for a list of apps that meet the guidelines in the Blackboard documents (the project guideline PDFs and the list of tools and apps from the instructor). I also asked what each tool does, what it needs and how much knowledge it requires, so I could compare them.

Sat, Oct 10

> I asked the AI for a work schedule for the project and asked it to help me decide among the three tools I had selected.

Fri, Oct 16

> I asked the AI to draft an email to the professor to schedule a meeting about the project. The email explained my tool ideas together with Steven's ideas.

Sat, Oct 17

> After the meeting with the professor, and after a discussion between Steven and me, I asked the AI whether certain ways of combining our two tools in one project were viable. Before the meeting I had also asked the AI for a list of questions to ask the professor.

Thu, Oct 22

> I asked the AI to create the GitHub repository following the project specifications, and I gave it access to the repository.

Sat, Oct 24

> I asked the AI to review the document so far and correct the spelling. Using voice chat, I also told the AI my ideas so it could build this Prompt History appendix, because my queries were in Spanish.

Project Contract

Everyone contributes to report writing and review. Roles may shift if workload changes, with the agreement of the team.

Communication. We will use Discord or WhatsApp for communication.

GitHub. Daniel owns the GitHub repository. We will work together to fill the folder and files accordingly to the project requirements.

Decision making. We aim for consensus. If we disagree, we vote by majority; if still unresolved, we ask the instructor.

Deadlines and work logs. Each member records dates, hours and tasks in the Work Log after each work session.

Accountability. Members who cannot attend or finish a task will tell the team as early as possible. Repeated missed work will first be discussed within the team and, if not resolved, reported to the instructor.

Use of AI tools. AI tools may be used for [e.g., organizing notes, improving wording], and every use is recorded in AI use and Appendix for Project Proposal and Individual Progress Report. Decisions and final content remain the team's own.

Agreement. By signing below, each member agrees to these terms.

Name	Signature	Date
Daniel Anaya Murcia	Daniel Anaya Murcia	24/10/2026
Steven Anderson		23/10/2026

Work Log Table

Steven's Work Log Table

Date	Number of Hours	Description of work
Oct 9	20~ minutes	1. Daniel notified me regarding the project since I sick and miss the class.
Oct 15	3 Hours 30~ minutes	1. Chat with Daniel regarding the project. 2. Checking the video and .pdf regarding the project. 3. Deciding which apps for the project. 4. Asking ChatGPT what's the recommended tools for the apps, and the reasoning behind it. 5. Correcting ChatGPT since it assumes I want to use one tools for both apps. 6. Refining the scope for each apps. 7. Extended research regarding tools recommendation by ChatGPT. 8. Telling ChatGPT the possibility of using rooted phone and using rooted access for the project. 9. Brainstorming with ChatGPT about other possible tools, apps, and integration with Daniel's choice.
Oct 16	1~ Hour	1. Telling Daniel my choice of apps and tools. 2. Making a quick proposal using ChatGPT for tomorrow's meeting.
Oct 17	1~ Hour	1. Project Proposal Meeting. 2. Adjust my proposal based on your suggestions.

Oct 24	2 Hours 50~ minutes	1. Remind Daniel to create GitHub repo and google drive folder to share for working on the proposal. 2. Start doing the proposal. 3. Generating a proper logo for the project proposal. 4. Letting ChatGPT know the improvements for the projects and my rooted phone specifications. 5. Make the proposal look professional and tidy.
Oct 25	1 Hour 30~ minutes	1. Continuing the proposal. 2. Filling references with ChatGPT help. 3. Uploading for Daniel to work on it. 4. Notify Daniel.

Daniel's Work Log Table

Date	Number of Hours	Description of work
Oct 9	3 Hours	1. Searched Google and read the project guidelines and the instructor's list of tools and apps to collect information. 2. Asked the AI for useful links where I could read more about the tools and apps. 3. Asked the AI for a list of apps that meet the guidelines in the Blackboard documents.
Oct 10	1 Hour	1. Asked the AI to make a work schedule for the project. 2. Compared that schedule with the schedules of my other projects this semester. 3. Asked the AI to make changes to organize my time better.
Oct 16	1 Hour	1. Asked the AI to write a draft email to the professor so we could arrange a meeting.
Oct 17	4 Hours	1. Told the AI what I understood from the conversation with the professor, and it advised me on which step to follow. 2. Following that advice, I searched different websites and YouTube videos to understand how the possible tools work. 3. After a lot of analysis, I decided to go with ALEAPP, because it was more manageable and it could probably be combined better with Steven's idea.
Oct 22	1 Hour 10 minutes	1. Created the GitHub repository. 2. Connected the AI to the repository so it could check that everything was correct. 3. Fixed the README so it matched what the project requires.
Oct 24	5 Hours	1. Steven sent me his part of the report, and I used it as an example to write mine. 2. Wrote my part of the report.

		3. Asked the AI to help me with certain parts of the report so they read better, such as the prompt history. 4. Reviewed the whole document to check that it was as good as possible.

Closing Thoughts

I've always enjoyed exploring and experimenting with technology, whether it's building and overclocking PCs, modifying games, upgrading and maintaining laptops, or rooting Android phones and installing custom software. However, despite being familiar with these technical activities, I've never really explored the cybersecurity and digital forensics side of technology.

Through this project, I'm excited to explore a different perspective on something I've always enjoyed working with. Rather than just understanding how devices and applications function, I want to learn how to investigate them, identify digital evidence, and understand what information can be recovered from user activities. I hope this project will not only expand my technical knowledge but also introduce me to practical forensic skills that I can apply to future projects or potentially my career – Steven.

Following what Steven said, I want to use this project mostly to learn about mobile forensic tools. It is an area that is maybe not my favorite one in cybersecurity, but it can be very useful for me in the future, because I want to work on a red team, and red teamers sometimes also have to find vulnerabilities in mobile applications.

I hope I can do this project as well as possible. This is my last semester at the college and it is a very busy one, but I will try to organize my time as best I can and keep in touch with my teammate so that everything goes well. – Daniel

References

- Amnesty International Security Lab. (n.d.). AndroidQF: Android Quick Forensics [Computer software]. GitHub. https://github.com/mvt-project/androidqf

- Brignoni, A. (n.d.). ALEAPP: Android Logs Events And Protobuf Parser [Computer software]. GitHub. https://github.com/abrignoni/ALEAPP

- Cozic, L. (n.d.). Joplin [Computer software]. Joplin. https://joplinapp.org/help/
- CyberDefenders. (n.d.). Android breach [Online lab walkthrough]. https://cyberdefenders.org/walkthroughs/androidbreach/

- Discord Inc. (n.d.). Download Discord to talk, play, and hang out. https://discord.com/download

- Discord Inc. (2026, September 25). Your Discord data package. Discord Support. https://support.discord.com/hc/en-us/articles/360004957991-Your-Discord-Data-Package

- Google. (n.d.). Android Debug Bridge (adb). Android Developers. https://developer.android.com/tools/adb

- Mozilla. (n.d.). Firefox for Android (Fenix project) [Computer software]. Firefox Source Docs. https://firefox-source-docs.mozilla.org/mobile/android/fenix/Home.html

- Reproducible Builds. (n.d.). Diffoscope: In-depth comparison of files, archives, and directories [Computer software]. https://diffoscope.org/

- SecurityRonin. (n.d.). SQLite-forensic: Read-only SQLite forensic toolkit [Computer software]. GitHub. https://github.com/SecurityRonin/sqlite-forensic

- SensePost. (n.d.). Objection: Runtime mobile exploration [Computer software]. GitHub. https://github.com/sensepost/objection

- Skylot. (n.d.). JADX: Dex to Java decompiler [Computer software]. GitHub. https://github.com/skylot/jadx

- Valve Corporation. (n.d.). Steam Mobile. Steam. https://store.steampowered.com/mobile